# UD3 · El navegador como parte de la app — Actividades

**Módulo · Desarrollo Web en Entorno Cliente (DEW)**

### Actividades de la unidad

Este documento reúne las diez actividades de la unidad, que en los apuntes aparecen intercaladas tras el apartado teórico que las motiva (el número del apartado figura en la tabla final). La A10 es el producto evaluable y se construye con piezas de todas las anteriores.

Todas las actividades se hacen en tu repositorio `dwec-ejercicios`, dentro de la carpeta `ut03`, una subcarpeta por actividad (`a1-objetos`, `a2-cadenas`…). La A10 va en `dwec-proyecto`. Haz un commit por actividad terminada y `push` al acabar cada sesión. Salvo que se indique lo contrario, las actividades son individuales.

A diferencia de la UD2, en esta unidad el `index.html` ya no está vacío: tus programas escribirán en la página. Este es el punto de partida de cada actividad (cambia el título y el `h1`):

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>UD3 · A1</title>
  <link rel="stylesheet" href="estilos.css">
  <script src="app.js" defer></script>
</head>
<body>
  <h1>UD3 · A1 · Objetos predefinidos</h1>
  <main id="salida"></main>
</body>
</html>
```

El elemento `<main id="salida">` es el «hueco» en el que escribirán tus programas (apartado 5). El fichero `estilos.css` es opcional: lo que importa en este módulo es el JavaScript, pero una hoja de estilos mínima hace más agradable ver el resultado.

> ***Importante.*** *Abre siempre las páginas con* ***Live Server*** *(`http://127.0.0.1:5500/...`), nunca haciendo doble clic en el fichero (`file://`). Varias cosas de esta unidad (cookies, `localStorage` compartido entre páginas, `postMessage`, `BroadcastChannel`) dependen del* ***origen*** *de la página, y con `file://` no funcionan o funcionan de forma distinta. Y usa siempre la misma dirección: `127.0.0.1:5500` y `localhost:5500` son* ***orígenes distintos*** *y no comparten almacenamiento.*

### A1. Explorando los objetos predefinidos (CE a, h)

**Objetivo:** identificar los objetos predefinidos disponibles, clasificarlos por familia y comprobar en qué entornos existen.

1. En la consola de las DevTools, para cada uno de estos nombres, anota su `typeof` y a qué familia pertenece (nativo del lenguaje, navegador o documento): `Math`, `Date`, `JSON`, `window`, `document`, `location`, `navigator`, `localStorage`, `console`, `setTimeout`, `alert`, `Intl`, `screen`, `history`.
2. Crea `ut03/a1-objetos` con el `index.html` de partida y un `app.js` que muestre por consola, con `console.table`, una tabla con al menos ocho de esos nombres y dos columnas: `tipo` (el resultado de `typeof`) y `familia`. Pista: de momento puedes construir la tabla con un objeto literal por fila, como hiciste con `console.table` en la UD2.

3. Ejecuta el mismo `app.js` con Node.js. Anota cuáles de esos nombres dan `ReferenceError` y explica por qué con la tabla del apartado 1.1. Modifica el programa para que **no falle** en Node: usa `typeof nombre` (que nunca lanza error) en lugar de acceder al nombre directamente.
4. Con `console.dir(navigator)` y el autocompletado, localiza tres propiedades de `navigator` que no aparezcan en estos apuntes y anota qué crees que indican. Comprueba tu hipótesis en MDN.
5. Averigua cuántas propiedades y métodos tiene `Math`: prueba primero `Object.keys(Math).length` y después `Object.getOwnPropertyNames(Math).length`, y anota por qué no coinciden (pista: busca «enumerable» en MDN). Con el autocompletado localiza el método que calcula el máximo de varios números y el que devuelve el valor absoluto. Comprueba `Math.max(3, 9, 4)` y `Math.abs(-7)`.

**Entrega:** la carpeta `a1-objetos` y un `NOTAS.md` con la clasificación del punto 1, los resultados del punto 3 y las respuestas del punto 4.

### A2. Laboratorio de cadenas y números (CE a, h)

**Objetivo:** aplicar los métodos de `String`, `Number` y `Math` para limpiar, transformar y generar valores.

Crea `ut03/a2-cadenas` con el `index.html` de partida y un fichero por punto (`1-nombres.js`, `2-slug.js`…), todos enlazados con `<script defer>`. Cada programa pide sus datos con `prompt` (todavía) y muestra el resultado por consola. Documenta con JSDoc las funciones que escribas.

1. Nombre normalizado. Pide un nombre y apellidos escritos de cualquier manera (`'  ada LOVELACE byron  '`) y muestra: el nombre con cada palabra en mayúscula inicial y el resto en minúscula, las iniciales (`'A. L. B.'`) y el número de palabras. Pista: `trim`, `split(' ')`, `at(0)`, `slice(1)`, un bucle `for…of`.
2. Slug y nombre de fichero. Pide un título y muestra su *slug* (apartado 2.2) y un nombre de fichero con la fecha de hoy delante, con el formato `2026-10-13-titulo-del-articulo.md`. Para la fecha usa de momento `new Date().toISOString().slice(0, 10)` (lo entenderás en el apartado 3).
3. Censura. Pide una frase y una palabra, y muestra la frase con todas las apariciones de esa palabra (sin distinguir mayúsculas) sustituidas por asteriscos de la misma longitud (`'*'.repeat(palabra.length)`). Pista: compara en minúsculas, pero conserva el original; `replaceAll` solo acepta texto literal, así que si no distingues mayúsculas tendrás que recorrer con `indexOf` en bucle o convertir todo a minúsculas antes.
4. Dados y porcentajes. Escribe una función `enteroAleatorio(min, max)` con el patrón del apartado 2.4 y úsala para simular 1000 tiradas de un dado de seis caras, contando cuántas veces sale cada cara (seis contadores) y mostrando el porcentaje de cada una con dos decimales. ¿Se acercan al 16,67 %?
5. Contraseña provisional. Genera una contraseña de 10 caracteres eligiendo al azar letras y dígitos de la cadena `'ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnpqrstuvwxyz23456789'` (se han quitado los caracteres que se confunden). Muestra también `crypto.randomUUID()` y explica en un comentario por qué el segundo es adecuado para identificadores y tu función no lo sería para contraseñas reales.
6. Redondeos. Muestra en una tabla (`console.table`) el resultado de `round`, `floor`, `ceil`, `trunc` y `toFixed(0)` para los valores `2.5`, `-2.5`, `2.4999` y `1.005`. Comenta en el código las dos filas que más te sorprendan.

**Entrega:** la carpeta `a2-cadenas` y `PRUEBAS.md` con una tabla de al menos tres casos de prueba (entrada · esperado · obtenido) para los puntos 1, 2 y 3.

### A3. Fechas e importes bien formateados (CE a, c, h)

**Objetivo:** operar con fechas y presentar fechas, números e importes localizados con `Intl`.

Crea `ut03/a3-fechas` con el `index.html` de partida y un fichero por punto.

1. Edad exacta. Pide la fecha de nacimiento en formato ISO (`AAAA-MM-DD`) y muestra la edad en años cumplidos (cuidado con quienes aún no han cumplido años este año), el día de la semana en que nació (con `Intl`, en español) y cuántos días faltan para el próximo cumpleaños. Valida que la fecha sea correcta: `new Date(texto)` con un texto inválido produce una fecha cuyo `getTime()` es `NaN`.
2. Cuenta de días. Muestra cuántos días faltan para el final de la primera evaluación (18 de diciembre de 2026) y para el final de las clases en el centro (5 de febrero de 2027), y exprésalo también con `Intl.RelativeTimeFormat` («dentro de 68 días»). Si la fecha ya pasó, debe decir «hace N días».
3. Mismo importe, cuatro países. Pide un importe y muestra, con `console.table`, cómo lo vería un usuario en `es-ES` (EUR), `en-US` (USD), `de-DE` (EUR), `ja-JP` (JPY) y `en-IN` (INR). Observa la agrupación de cifras en `en-IN` y los decimales en `ja-JP`.
4. Tabla de precios localizada. Con cinco precios fijos en una cadena separada por comas (`'12.5,8,1250,0.99,19999'`), muestra una tabla con cada precio formateado en euros, con IGIC del 7 % aplicado (también formateado) y el porcentaje de impuesto formateado con `style: 'percent'`. Crea un solo formateador de moneda y reutilízalo.
5. El idioma del usuario. Muestra `navigator.language` y `navigator.languages`, y vuelve a formatear la fecha de hoy con `dateStyle: 'full'` usando ese idioma. Cambia el idioma preferido del navegador en su configuración (o usa otro navegador), recarga y anota qué cambia.

**Entrega:** la carpeta `a3-fechas` y `PRUEBAS.md` con tres casos de prueba para el punto 1 (incluye uno de alguien que cumple años hoy y uno con una fecha inválida).

### A4. Reloj y cuenta atrás (CE a, c, h)

**Objetivo:** programar tareas periódicas y diferidas, cancelarlas y combinarlas con `Intl`.

Crea `ut03/a4-tiempo` con el `index.html` de partida. En esta actividad el resultado se muestra en la página: de momento, usa `document.getElementById('salida').textContent = texto;` para escribir texto dentro del `<main>` (lo explicamos en el apartado 5; aquí basta con copiarlo).

1. Reloj. Muestra la hora actual en la página con formato `HH:MM:SS` (`Intl.DateTimeFormat` con `hour`, `minute` y `second` a `'2-digit'`) y actualízala cada segundo. Añade debajo la fecha completa (`dateStyle: 'full'`) en el idioma del navegador. Comprueba con la pestaña Performance o simplemente observando que el reloj no «salta» dos segundos de golpe.
2. Cuenta atrás con cancelación. Pide con `prompt` un número de segundos y haz una cuenta atrás en la página; al llegar a cero muestra «¡Tiempo!» y cancela el intervalo. Comprueba con un `console.log` dentro de la función que, tras el cero, deja de ejecutarse.
3. El orden de las cosas. Escribe un programa con cinco `console.log` y tres `setTimeout` con retardos `0`, `100` y `50`, mezclados en cualquier orden. Escribe en un comentario, antes de ejecutarlo, el orden de salida que predices; ejecútalo y anota el real.
4. Trampa. Escribe `setTimeout(saludar(), 1000)` con una función `saludar` que escribe en consola. Anota qué ocurre, por qué, y corrígelo de las dos formas posibles (sin paréntesis y con función flecha envolvente).

5. Ampliación · Pomodoro. Un temporizador de 25 minutos de trabajo y 5 de descanso que alterna automáticamente, muestra el tiempo restante como `MM:SS` y cambia el `document.title` para que se vea el tiempo en la pestaña aunque no esté activa. Para probarlo, parametriza las duraciones en segundos.

**Entrega:** la carpeta `a4-tiempo` con un fichero por punto y las predicciones del punto 3 como comentario en el código.

### A5. La página habla (CE c, d, h)

**Objetivo:** generar texto y HTML desde código de forma segura, y empezar a aplicar los datos del proyecto.

Crea `ut03/a5-pagina` con el `index.html` de partida y una hoja `estilos.css` mínima (al menos una regla para `[data-tema="oscuro"]` que cambie los colores de fondo y texto del `body`).

1. **Tres formas de escribir.** Inserta el texto `<b>Hola</b>` en el `<main>` con `textContent` y luego con `innerHTML`, y después añade un párrafo con `insertAdjacentHTML('beforeend', …)` sin borrar lo anterior. Haz una captura de las tres y explica en `NOTAS.md` la diferencia.
2. **Ficha del proyecto.** Con los datos que ya tenías en variables en la A9 de la UD2 (o con cuatro o cinco propiedades de un elemento de tu colección), genera con una plantilla literal una «tarjeta» en HTML (`<article>` con un `<h2>` y una lista de definición `<dl>`) y píntala en el `<main>`. Formatea con `Intl` cualquier fecha o importe que contenga.
3. **Lista generada.** Escribe en una cadena separada por comas los nombres de cinco elementos de tu colección y genera una `<ul>` con un `<li>` por elemento, acumulando el HTML en un bucle y asignándolo una sola vez.
4. **El ataque.** Pide con `prompt` el nombre del usuario y salúdalo con `innerHTML` dentro de un `<h2>`. Escribe como nombre `<img src=x onerror="alert('XSS')">` y comprueba qué pasa. Corrígelo de las **dos** formas del apartado 5.3 (en dos ficheros distintos) y comprueba que el ataque ya no funciona. Incluye `escaparHtml` documentada con JSDoc.
5. **Tema y título.** Añade un botón `<button id="tema">Cambiar tema</button>` al HTML. Para que haga algo necesitas una línea de la UD7, que de momento copias tal cual: `document.getElementById('tema').addEventListener('click', () => { … });` («cuando hagan clic, ejecuta esto»). Dentro, alterna `document.body.dataset.tema` entre `'claro'` y `'oscuro'` y actualiza `document.title` con el tema activo.

**Entrega:** la carpeta `a5-pagina` con un fichero por punto, `estilos.css` y `NOTAS.md`.

### A6. Radiografía del navegador (CE b, c, h)

**Objetivo:** analizar los objetos del BOM y usarlos para leer la URL y adaptar la página al navegador del usuario.

Crea `ut03/a6-bom` con el `index.html` de partida y una hoja de estilos con tema claro y oscuro.

1. Panel de información. Genera en el `<main>` una tabla HTML (`<table>`) con estas filas, leídas del BOM: idioma, idiomas, conexión, núcleos de CPU, pantalla táctil, tamaño de la pantalla, tamaño de la zona visible, orientación, origen de la página, ruta y si prefiere el tema oscuro. Usa `textContent` o `escaparHtml` para los valores.
2. Parámetros de la URL. Lee con `URLSearchParams` los parámetros `nombre` y `tema`. Si viene `nombre`, salúdalo en un `<h2>` (de forma segura: pruébalo con una etiqueta). Si viene `tema` (`claro` u `oscuro`),

aplícalo con `dataset.tema`; si no viene, usa el del sistema con `matchMedia`. Prueba `index.html?nombre=Ada&tema=oscuro`.

3. Navegación. Añade tres botones: «Añadir parámetro» (navega con `location.href` a la misma página con `pagina` incrementado en uno, construyendo la cadena con `URLSearchParams`), «Atrás» (`history.back()`) y «Recargar». Muestra `history.length` y comprueba cómo crece. Añade un cuarto que use `location.replace` y explica en `NOTAS.md` qué cambia en el botón Atrás del navegador.
4. Pantalla completa y título. Un botón que ponga la página en pantalla completa y otro que la saque; mientras está en pantalla completa, el `document.title` debe empezar por `[PC]`. Comprueba qué ocurre si llamas a `requestFullscreen()` sin un clic (desde el script directamente) y anótalo.
5. Detección de características. Comprueba con `if` si existen `navigator.share`, `navigator.clipboard` y `navigator.geolocation`, y muestra en el panel «disponible» o «no disponible» para cada una. Añade un botón «Copiar URL» que copie `location.href` al portapapeles (solo si la API existe) y confirme en la página que se ha copiado.

**Entrega:** la carpeta `a6-bom` y `NOTAS.md` con las respuestas de los puntos 3 y 4.

### A7. Diálogos propios (CE e, h)

**Objetivo:** interactuar con el usuario con los diálogos del navegador y sustituirlos por diálogos propios con `<dialog>`.

Crea `ut03/a7-dialogos` con el `index.html` de partida.

1. Los tres de siempre. Un programa que, al cargar, pide un nombre con `prompt` (con valor por defecto), confirma con `confirm` si quiere continuar y termina con `alert`. Trata los tres casos de `prompt` (texto, vacío, cancelación) mostrando mensajes distintos en la página. Comprueba qué devuelven `confirm` y `prompt` si marcas en el navegador «no permitir más diálogos» (Chrome lo ofrece tras varios seguidos).
2. Confirmar con `<dialog>`. Un botón «Vaciar lista» que abre un diálogo modal de confirmación como el del apartado 7.2. Si se confirma, vacía el `<main>` y muestra un mensaje de estado; si se cancela (botón o Esc), no hace nada. Da estilo al diálogo y a `::backdrop` en `estilos.css`.
3. Pedir un dato con `<dialog>`. Un botón «Añadir» que abre un diálogo con un `<input>` para el nombre de un elemento de tu colección y, al aceptar, lo añade como `<li>` a una lista en la página (de forma segura). Al cancelar, no añade nada. Comprueba que `required` impide aceptar con el campo vacío.
4. Toast. Una función `avisar(texto)` que escribe el texto en un `<p id="mensaje" role="status">` y lo borra a los tres segundos; si se llama dos veces seguidas, debe cancelar el borrado anterior (guarda el identificador del `setTimeout` en una variable externa). Úsala en los puntos 2 y 3.
5. Ampliación · Notificación. Un botón que pide permiso para notificaciones y, si se concede, muestra una notificación del sistema con el título de la página.

**Entrega:** la carpeta `a7-dialogos` con `estilos.css` y las respuestas del punto 1 en `NOTAS.md`.

### A8. Ventanas que se hablan (CE b, f, h)

**Objetivo:** abrir y controlar ventanas, y comunicar ventanas y pestañas con `postMessage` y `BroadcastChannel`.

Crea `ut03/a8-ventanas` con dos páginas: `index.html` (principal) y `previa.html` (secundaria), cada una con su script.

1. Abrir y controlar. En la principal, tres botones: «Abrir vista previa» (abre `previa.html` como ventana emergente de 480 × 640 con el nombre `previa`), «Enfocar» (`focus()` sobre la referencia) y «Cerrar» (`close()`). Muestra en la página si la ventana está abierta o cerrada (`closed`), actualizándolo cada segundo con `setInterval`. Intenta abrirla desde un `setTimeout` al cargar la página y anota en `NOTAS.md` qué hace el navegador.
2. Enviar una ficha. Un botón «Enviar» que manda a la vista previa, con `postMessage`, un objeto con el título, el precio y la fecha de un elemento de tu colección. La vista previa lo muestra formateado con `Intl` (de forma segura) y responde con un mensaje de confirmación que la principal escribe en su zona de estado. Comprueba el origen en ambos lados.
3. Origen equivocado. Cambia en el emisor el origen de destino por `'http://localhost:5500'` (si trabajas en `127.0.0.1`) y observa en la consola qué ocurre. Restáuralo y explica en `NOTAS.md` por qué el navegador se ha negado.
4. Tema sincronizado. Abre `index.html` en dos pestañas. Un botón «Cambiar tema» que alterna `dataset.tema` y lo emite por un `BroadcastChannel` llamado `preferencias`; la otra pestaña debe cambiar sola. Comprueba que la pestaña emisora no recibe su propio mensaje.
5. Ampliación · `noopener`. Abre `https://developer.mozilla.org` con y sin `'noopener'` y comprueba en la consola de la nueva pestaña el valor de `window.opener` en cada caso.

**Entrega:** la carpeta `a8-ventanas` con las dos páginas y `NOTAS.md`.

### A9. Guardar y recuperar (CE g, h)

**Objetivo:** usar cookies, `localStorage` y `sessionStorage`, elegir el adecuado en cada caso y depurar con la pestaña Application.

Crea `ut03/a9-almacen` con el `index.html` de partida y `almacen.js` con las funciones `guardar` y `cargar` del apartado 9.2 (enlázalo antes que `app.js`).

1. Contador de visitas, tres veces. Guarda y muestra «Has visitado esta página N veces» usando una cookie (con `max-age` de un año), `localStorage` y `sessionStorage`, los tres a la vez con claves distintas. Recarga varias veces, abre la página en otra pestaña y después cierra y abre el navegador. Anota en `NOTAS.md` qué contador cambia en cada caso y por qué.
2. Cookies con atributos. Crea una cookie de sesión (sin `max-age`) y otra con `max-age=60`. Obsérvalas en Application → Cookies (columnas Expires y Size), espera un minuto y recarga. Intenta leer desde JavaScript una cookie marcada como `HttpOnly` (puedes crearla a mano en Application) y anota qué ves en `document.cookie`.
3. Guardar un objeto. Guarda en `localStorage` un objeto `preferencias` con cuatro propiedades (una de ellas numérica y otra booleana) con `guardar`, recupéralo con `cargar` y muestra en la página cada propiedad con su `typeof`. Después guárdalo sin `JSON.stringify` y mira qué aparece en Application; recupéralo y explica en `NOTAS.md` qué has obtenido.
4. Datos corruptos. Edita a mano en Application el valor de `preferencias` para que deje de ser JSON válido (borra una llave) y recarga. Comprueba que `cargar` devuelve el valor por defecto y que la página no se rompe. Quita el `try…catch` y comprueba qué pasa; vuelve a ponerlo.
5. Dos pestañas. Añade un `addEventListener('storage', …)` que escriba en la página «La otra pestaña ha cambiado la clave X» cuando cambie algo. Abre dos pestañas y modifica las preferencias en una.

6. Ampliación · IndexedDB. Ejecuta el ejemplo del apartado 9.4, localiza la base de datos en Application → IndexedDB y añade un segundo objeto a mano con `put` desde la consola.

**Entrega:** la carpeta `a9-almacen` con `almacen.js`, un fichero por punto y `NOTAS.md`.

### A10. Preferencias de tu proyecto (producto de la unidad; CE a–h)

**Objetivo:** construir la primera versión del requisito R6 del proyecto: una página de preferencias que se guarda y se recupera del navegador, con aviso de consentimiento, formato localizado y un reloj en la página de inicio.

Se hace en `dwec-proyecto`, en una página nueva `preferencias.html` (con su `preferencias.js`, `almacen.js` y la hoja de estilos del proyecto), y se modifica `index.html` para integrar el reloj y aplicar las preferencias. Las piezas ya las has construido en A3 a A9: aquí las ensamblas.

**Requisitos (R = obligatorio, O = opcional):**

| | Requisito |
|---|---|
| R1 | La página de preferencias muestra cuatro controles: tema (claro / oscuro / sistema), idioma (al menos `es-ES`, `en-GB` y otro), moneda (al menos EUR, USD y otra) y elementos por página (5, 10, 20). Puedes usar `<select>`; para leer su valor basta con `document.getElementById('tema').value`. |
| R2 | Al pulsar «Guardar», las preferencias se guardan en `localStorage` como un único objeto JSON con `version`, mediante las funciones `guardar` y `cargar` de `almacen.js`, documentadas con JSDoc y con `try…catch`. Un `@typedef Preferencias` documenta la estructura. |
| R3 | Al cargar cualquier página del proyecto, se leen las preferencias guardadas (o las de por defecto) y se aplican: el tema con `dataset.tema` (si es «sistema», con `matchMedia`), el atributo `lang` del `<html>` y el resto queda disponible para las unidades siguientes. |
| R4 | Una vista previa en la página de preferencias muestra, con las preferencias actuales, un importe de ejemplo (`1234.5`) formateado en la moneda e idioma elegidos, la fecha de hoy con `dateStyle: 'full'` y un texto con `Intl.RelativeTimeFormat` («hace 3 días»). Se actualiza al guardar. |
| R5 | Aviso de consentimiento según el apartado 10.3: aparece solo si no se ha respondido, con «Aceptar» y «Rechazar» igual de visibles y un enlace a un texto que explica qué se guarda. Si se rechaza, las preferencias se aplican pero no se persisten (se mantienen en `sessionStorage` mientras dure la pestaña). |
| R6 | Un botón «Borrar mis datos» que elimina todo lo guardado por el proyecto (`localStorage.clear()` y `sessionStorage.clear()`), vuelve a las preferencias por defecto y muestra de nuevo el aviso. Un mensaje de estado (*toast*, A7) confirma cada acción. |
| R7 | La página de inicio muestra un reloj con la hora actual (`HH:MM:SS`) y la fecha larga en el idioma de las preferencias, actualizado cada segundo. |
| R8 | El `README.md` incluye la tabla «Datos que se guardan en el navegador» (apartado 11.2) y una captura de la pestaña Application con las claves del proyecto. |
| O1 | Las preferencias se sincronizan entre pestañas con el evento `storage` o con `BroadcastChannel`: al guardar en una, la otra cambia el tema sin recargar. |
| O2 | Un botón «Vista previa en ventana» abre una ventana emergente con la página de inicio y le envía las preferencias con `postMessage` para que las aplique. |
| O3 | Un selector de zona horaria (`Intl.DateTimeFormat` admite la opción `timeZone`) y el reloj muestra la hora de la zona elegida. |

**Entrega:** commit y push en `dwec-proyecto` con el mensaje `UD3: preferencias persistentes (R6)`. Ten preparado el proyecto para enseñarlo: se te pedirá que expliques una parte del código.

### Resumen de actividades y entregas

| Actividad | Apartado | Sesión orientativa | CE | Entrega |
|---|---|---|---|---|
| A1. Explorando los objetos predefinidos | 1 | 1 | a, h | `a1-objetos` + `NOTAS.md` |
| A2. Laboratorio de cadenas y números | 2 | 1–2 | a, h | `a2-cadenas` + `PRUEBAS.md` |
| A3. Fechas e importes bien formateados | 3 | 2 | a, c, h | `a3-fechas` + `PRUEBAS.md` |
| A4. Reloj y cuenta atrás | 4 | 3 | a, c, h | `a4-tiempo` |
| A5. La página habla | 5 | 4 | c, d, h | `a5-pagina` + `NOTAS.md` |
| A6. Radiografía del navegador | 6 | 4–5 | b, c, h | `a6-bom` + `NOTAS.md` |
| A7. Diálogos propios | 7 | 5 | e, h | `a7-dialogos` + `NOTAS.md` |
| A8. Ventanas que se hablan | 8 | 6 | b, f, h | `a8-ventanas` + `NOTAS.md` |
| A9. Guardar y recuperar | 9 | 6–7 | g, h | `a9-almacen` + `NOTAS.md` |
| A10. Preferencias de tu proyecto | 10–11 | 7–8 | a–h | `dwec-proyecto` (R6) |
| Prueba práctica | — | 8 | a–h | En ordenador |